

Phase 2 Evaluation — Aurua

Student: Jorge Urias

Score: 86 / 100

Dimension	Score	Max
Architecture and Role Design	23	25
Tools, Memory, Data, and State Design	17	20
Prototype Quality	13	15
Evaluation Plan and Test Design	17	20
Risk and Governance Thinking	8	10
Documentation and Contribution Update	8	10
Total	86	100

Dimension Notes

Architecture and Role Design — 23/25. Phase 1 flagged a linear pipeline of five agents with no feedback loops, no cyclic routing, and no adaptive behavior. Phase 2 responds with a redesigned topology: four agents plus a deterministic renderer, a verifier-driven retry loop, and a parallel branch for the coding and writing tasks. Each agent now has an explicit justification against a single-prompt baseline rather than a one-sentence assertion. The Pydantic state schema in `state.py` is implemented and typed, not just described, and the coordination logic in §3.3 specifies stopping conditions, retry budget, and the HITL checkpoint at the precision the rubric requires. One flag for Phase 3: the orchestrator in `src/run.py` is custom-built with no named framework. That is a legitimate engineering choice for a verify-revise-retry loop that does not map cleanly onto graph-based frameworks, but course requirements ask you to name the coordination approach explicitly and justify it against available alternatives.

Tools, Memory, Data, and State Design — 17/20. FAISS retrieval is implemented, exercised in every multi-agent run, and its configuration is version-controlled in `VERSION_NOTES.md` at the level of chunk size, overlap, top-k, and embedding model. The Pydantic models in `state.py` are typed end-to-end with accumulation semantics and field-level constraints (`duration_sec` bounded 5 to 30, `confidence` bounded 0 to 1), and the cost tracker is instrumented per run. The tool-to-agent table in §4.1 maps every Phase 3 tool to the agent that will use it — that is the right way to document planned work at Phase 2, and it is not penalized here. The three-point gap comes from one design choice that is not yet resolved in the implemented portion: because all fields live in a single shared `RunState`, there is no explicit access boundary preventing an agent from reading fields it has no business reading. Specifying per-agent read access in Phase 3 would close that.

Prototype Quality — 13/15. The TC1 trace is a genuine captured execution: timestamps on 2026-04-16 at 17:29:19 UTC with per-node latencies, real FAISS span IDs and retrieval scores, actual verifier rejection reasoning that quotes specific span content, and a planner revision that addressed exactly the flagged claim. TC3 and TC4 show the vague-input short-circuit running in under four seconds with specific retrieval scores (0.17-0.20) as the scope signal. The TC6 single-prompt baseline is a real run with verbatim source quotes confirmed against the transcript line by line. The 13/15 reflects what Phase 2 set out to build and demonstrates: the core agentic loop from intent through verification, including retry logic and the baseline ablation. The remaining two points are reserved for Phase 3, when the rendering branch is complete and an end-to-end run can be evaluated from question to video.

Phase 2 Evaluation — Aurua

Evaluation Plan and Test Design — 17/20. Phase 1 noted that all five success criteria were qualitative with no thresholds, test sets, or baselines. Phase 2's §6.2 addresses each of those gaps directly: grounding rate at 90%, confusion-type accuracy at 7 of 10, preference at 3 of 5 reviewers, each with a named measurement method. Beyond meeting the Phase 1 ask, running TC6 as a principled ablation and reporting that the single-prompt baseline outperformed the multi-agent pipeline on the primary anchor case is the strongest evaluation decision in this submission. The failure log documents three failures with root-cause analysis, evidence pointers to specific trace files, and concrete fixes planned for Phase 3. The remaining three points: the evaluation covers only Agents 1-3 because the rendering branch is Phase 3 scope; the preference test with five reviewers is defined but not yet run; and TC6 ran once rather than the two runs per case called for in §6.3.

Risk and Governance Thinking — 8/10. The risk table is architecture-specific throughout. The Grounding Verifier is the primary hallucination mitigation and TC3 is its evaluation case. The retry budget is a hard governance cap and the cost tracker is instrumented in every run. The boundary-refusal behavior is tested, not just promised. The governance section in §7.1 specifies reproducibility via temperature-0 determinism plus state.json serialization, which is the right level of specificity for Phase 2. The gap is that the disclaimer scene and per-scene citation in the rendered output remain planned rather than implemented, so their effectiveness is not yet verifiable.

Documentation and Contribution Update — 8/10. AI_USAGE.md has three dated entries with specific prompt excerpts, and the entries correctly distinguish what was delegated (document drafting, evaluation artifact generation) from what was independently verified (spot-checked line numbers in the transcript, FAISS documentation, Pydantic API). VERSION_NOTES.md records the exact retrieval configuration per run so results can be reproduced. The contribution update in §8 has a week-by-week Phase 3 timeline with named deliverables. The two-point gap: the checkboxes in the "What was changed manually" section of Entry 02 are unchecked, which means the review steps are listed as to-do items rather than completed steps. Course policy treats incomplete disclosure tracking as a documentation quality issue, even when the underlying actions appear to have been taken.

What Worked

Phase 1 scored this project 65/100 with three specific action items: anchor to a concrete subject and scenario, justify why multiple agents are needed over a single LLM call, and add quantitative success criteria with an AI usage disclosure. Phase 2 addresses all three, and does so precisely rather than generically. The anchor is 3B1B's backpropagation Ch3-4 with five test cases built around it. Each agent's justification is now an argument about what degrades without it, not a statement that the tasks are distinct. Success criteria have specific thresholds and measurement methods. The AI usage disclosure has three dated entries with prompt logs. Responding to Phase 1 feedback at that level of precision is how Phase 2 builds on Phase 1 rather than repeating it.

The TC6 ablation stands on its own. AI_USAGE.md Entry 01 records that the Phase 1 document contained a fabricated hallucination rate that Jorge flagged as a TODO before running actual tests. Phase 2 replaces it with real comparative data showing the baseline outperforming the pipeline on the primary anchor case, documents the root cause in the failure log, and explains the architectural implication without minimizing it. The Pydantic state schema in state.py reinforces the same principle: the architecture claim and the implementation agree, with typed field-level constraints and accumulation semantics that the orchestrator uses directly.

3 Things to Address in Phase 3

1. **Wire Agents 4a and 4b and produce at least one rendered video.** Phase 3 requires a final artifact and Aurua's central claim is that it generates grounded animated explainers. The prompts for the Animation Coder and Narration Writer are defined and the orchestrator has the verified scene plan ready to hand off. At minimum, wire both agents for TC1's verified plan and run the Renderer for one scene to produce a rendered clip, even if TTS and ffmpeg muxing are not complete for the full video. Without any rendered output, the final artifact requirement cannot be evaluated.

Phase 2 Evaluation — Aurua

2. **Apply the v0.3 retrieval fixes to TC2 and TC5 and report the results honestly.** The failure analysis is excellent as written, and Phase 3 requires you to show what changed after testing, not just what failed. The three fixes are specific and testable: smaller chunks, higher top-k, and query expansion in Agent 1. Run TC2 and TC5 under v0.3 and record whether the direct-answer span now appears in retrieval's top results. If the fix closes the gap, the architectural argument is substantially stronger. If it only partially closes it, document that honestly and note what further work is needed, which is exactly the failure analysis format the rubric rewards.
3. **Name and justify the orchestration approach relative to available frameworks.** Phase 3 documentation requires you to specify the orchestration framework used. If you are continuing with the custom orchestrator in `src/run.py`, write a short justification in the Phase 3 report that compares it against at least one graph-based framework (LangGraph is the most obvious candidate for a verify-retry loop) and explains why the custom approach was the right choice for this pipeline. If you migrate to a framework, name it, version it in `VERSION_NOTES.md`, and update the architecture diagram. Either path is fine; the requirement is that the coordination layer is explicitly named and justified, not left implicit.